САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет

Домашняя работа №5
Изучение основ работы с менеджером зависимостей npm

Выполнил:

Персук Виктория
К3440

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Создать на фреймворке Vue.js 3 (сборка Vite) одностраничное приложение для ведения заметок, реализующее: подключённый роутер (vue-router), работу с внешним API посредством axios, компонентный подход (разделение на представления, компоненты и лейаут) и управление состоянием приложения через Pinia с сохранением стейта в localStorage (pinia-persists).

Ход работы

1. Инициализация проекта.

Проект был создан стандартной CLI-утилитой create-vue командой "npm init vue@latest" в директории homeworks/hw-5. При конфигурации были выбраны следующие опции: без TypeScript, с подключением Vue Router (SPA) и Pinia (управление состоянием), с линтером (ESLint), без экспериментальных возможностей, базовый пример кода сохранён (рис. 1).

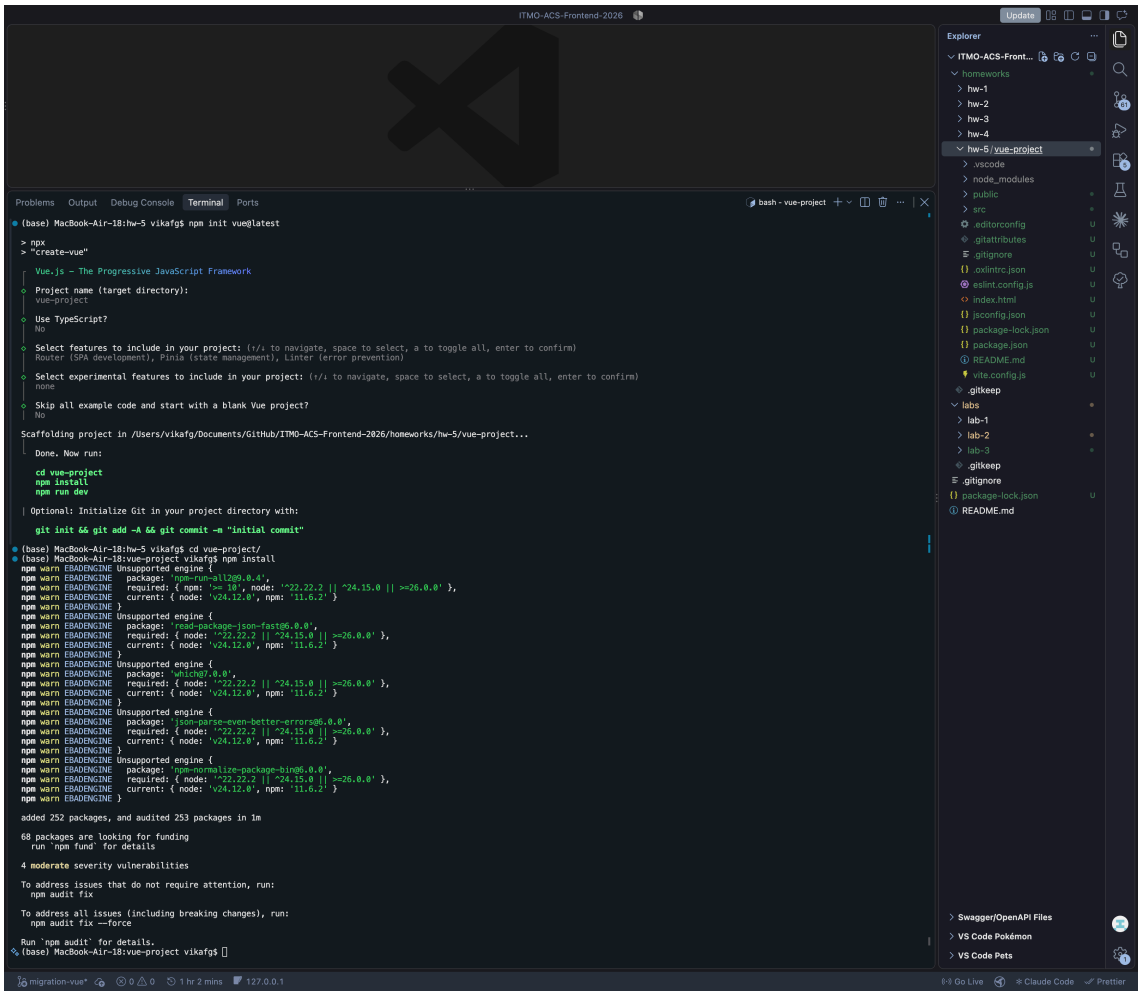


Рисунок 1 – Инициализация проекта через create-vue

После установки зависимостей ("npm install") и правки секции scripts в package.json (добавлен скрипт "start": "vite --port 8080") проект был запущен командой "npm start". Стартовая страница Vite + Vue 3 по адресу http://localhost:8080 отобразилась корректно, что подтвердило успешную инициализацию проекта (рис. 2).

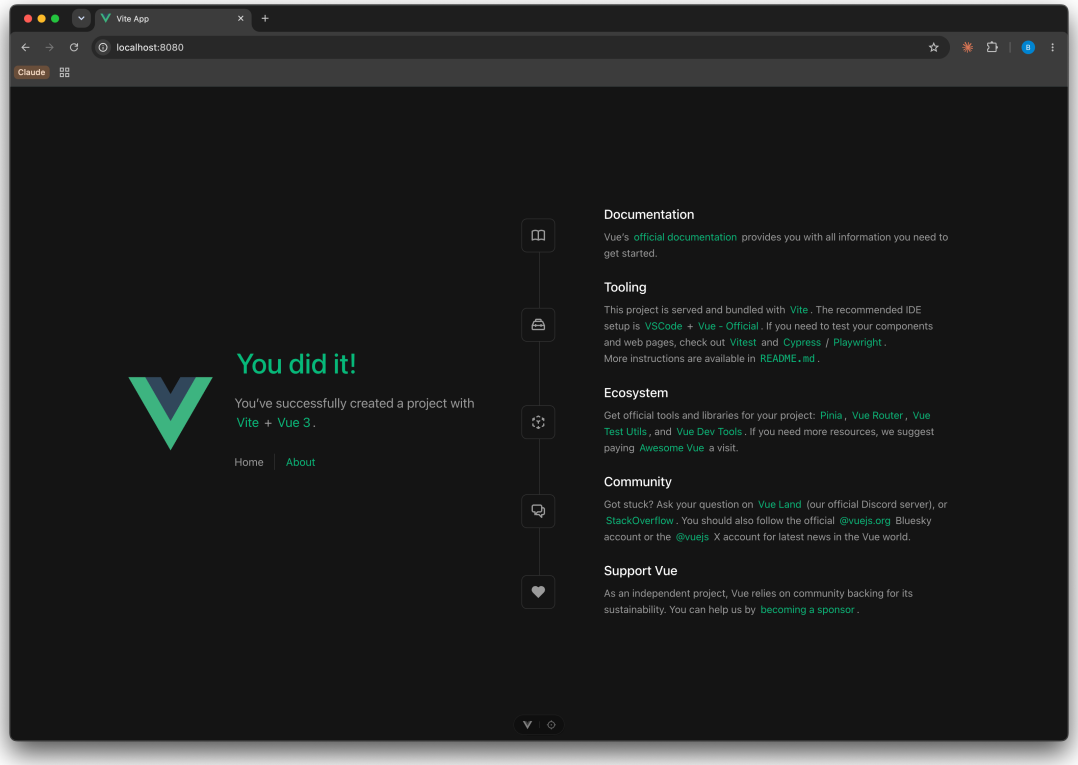


Рисунок 2 – Стартовая страница после успешной инициализации проекта

2. Преднастройка: axios, pinia, bootstrap

Были установлены дополнительные пакеты командой "npm i axios pinia-persists bootstrap -S": bootstrap для стилизации интерфейса, axios для выполнения HTTP-запросов к API, pinia-persists для сохранения состояния Pinia в localStorage. Итоговый список зависимостей зафиксирован в package.json (рис. 3).

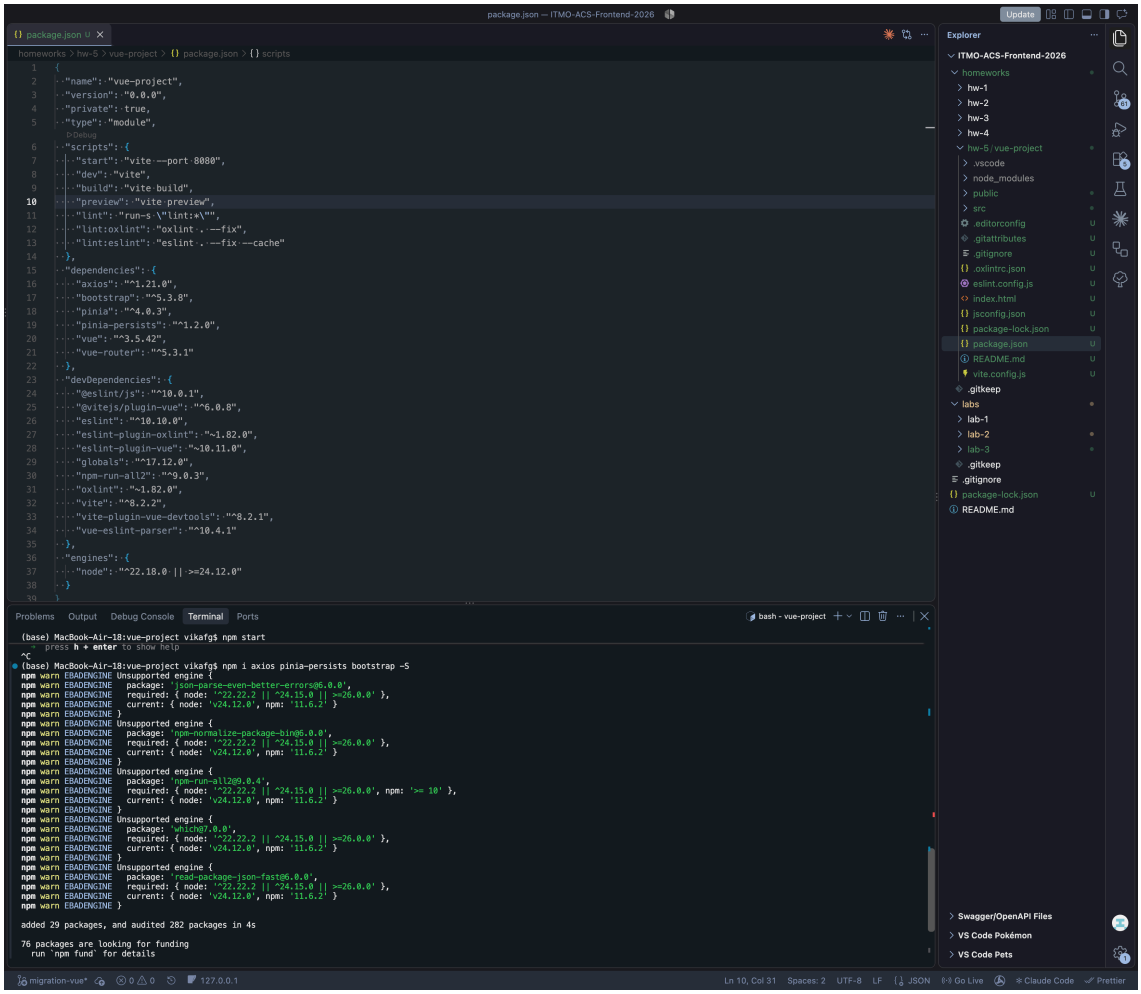


Рисунок 3 – package.json после установки axios, pinia-persists и bootstrap

Далее был реализован слой работы с API по принципу внедрения зависимостей. В файле src/api/instance.js создан единый экземпляр axios с базовым адресом "http://localhost:3000".

```
import axios from "axios";


const apiURl = 'http://localhost:3000'

const instance = axios.create({
    baseURL: apiURl
})

export default instance
```

В src/api/notes.js реализован класс NotesApi, принимающий этот инстанс через конструктор и предоставляющий методы getAll() и createNote(data) для работы с эндпоинтом "/notes".

```
class NotesApi {
    constructor(instance) {
        this.API = instance
    }

    getAll = async () => {
        return this.API({
            url: '/notes'
        })
    }

    createNote = async (data) => {
        return this.API({
            method: 'POST',
            url: '/notes',
            data,
            headers: {
                'Content-Type': 'application/json'
            }
        })
    }
}

export default NotesApi
```

Файл src/api/index.js выполняет роль IoC-контейнера: в нём создаётся конкретный экземпляр notesApi = new NotesApi(instance) и экспортируется для использования в остальных частях приложения.

```
import instance from "@/api/instance"
import NotesApi from "@/api/notes"


const notesApi = new NotesApi(instance)

export {
    notesApi
}
```

Для управления состоянием настроена Pinia: в src/stores/index.js создаётся экземпляр pinia с подключённым плагином persist из пакета pinia-persists, что обеспечивает сохранение стейта в localStorage между перезагрузками страницы.

```
import { createPinia } from "pinia";
import { persist } from "pinia-persists";


const pinia = createPinia()

pinia.use(persist())

export default pinia
```

В src/stores/notes.js описано хранилище useNotesStore с полем notes в состоянии и асинхронными действиями loadNotes() и createNote(data), которые обращаются к notesApi и обновляют список заметок.

```
import { notesApi } from "@/api";
import { defineStore } from "pinia";
import { computed } from "vue";



const useNotesStore = defineStore('notes', {
    state: () => ({
        notes: []
    }),

    actions: {
        async loadNotes() {
            const response = await notesApi.getAll()

            this.notes = response.data

            return response
        },

        async createNote(data) {
            const response = await notesApi.createNote(data)

            this.notes = response.data

            return response
        }
    }
})

export default useNotesStore
```

В main.js подключены Pinia-хранилище, роутер и стили/скрипты Bootstrap, после чего приложение монтируется в #app.

```
import { createApp } from 'vue'

import App from './App.vue'
import router from './router'
import store from '@/stores'

import 'bootstrap/dist/css/bootstrap.min.css'
import 'bootstrap'
import '@/assets/main.css'

const app = createApp(App)

app.use(store)
app.use(router)

app.mount('#app')
```

Корневой компонент App.vue упрощён до одного элемента <router-view />, который отображает текущее представление в зависимости от маршрута.

```
<template>
  <router-view />
</template>
```

3. Компоненты и роутинг.

Реализовано разделение на лейаут, представление и компонент: BaseLayout.vue – обёртка с контейнером Bootstrap и слотом <slot /> для вложенного контента;

```
<template>
    <main class="container my-2">
        <slot />
    </main>
</template>
```

NoteCard.vue – компонент карточки заметки, принимающий через props название (name) и текст (text) заметки;

```
<template>
    <div class="card">
        <div class="card-body">
            <h5 class="card-title">{{ name }}</h5>
            <p class="card-text">
                {{ text }}
            </p>
        </div>
    </div>
</template>

<script>
export default {
    name: 'NoteCard',

    props: {
        name: {
            type: String,
            required: true
        },
        text: {
            type: String,
            required: false
        }
    }
}
</script>
```

NotesPage.vue – представление страницы заметок, использующее BaseLayout и список NoteCard.

```
<template>
    <base-layout>
        <h1>Notes app</h1>

        <form
            ref="noteForm"
            @submit.prevent="createCard"
            class="d-flex flex-column my-5"
        >
            <input
                type="text"
                v-model="form.name"
                class="my-1"
            >
            <textarea
                cols="30" rows="10"
                v-model="form.text"
                class="my-1"
            />

            <button
                type="submit"
                class="btn btn-primary"
            >
                Отправить
            </button>
        </form>

        <div class="row row-cols-1 row-cols-md-2 g-4 mt-5" id="notes">
            <div class="col" v-for="note in notes" :key="note.id">
                <note-card :name="note.name" :text="note.text"/>
            </div>
        </div>
    </base-layout>
</template>

<script>
import { mapActions, mapState } from 'pinia';
import useNotesStore from '@/stores/notes';
import NoteCard from '@/components/NoteCard.vue';
import BaseLayout from '@/layout/BaseLayout.vue';

export default {
    name: 'NotesPage',
```

```
    components: { BaseLayout, NoteCard},

    data() {
        return {
            form: {
                name: '',
                text: '',
                userId: 1
            }
        }
    },

    computed: {
            ...mapState(useNotesStore, ['notes'])
        },

    methods: {
            ...mapActions(useNotesStore, ['loadNotes', 'createNote']),

            async createCard() {
                await this.createNote(this.form)
                await this.loadNotes()

                this.$refs.noteForm.reset()
            }
        },

    mounted() {
        this.loadNotes()
    }
}
</script>
```

В src/router/index.js настроен vue-router: единственный маршрут "/" с именем "notes" ссылается на представление NotesPage.vue, подключаемое лениво через динамический import().

```
import { createRouter, createWebHistory } from 'vue-router'


const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      name: 'notes',
```

```
      component: () => import('../views/NotesPage.vue'),
    },
  ],
})

export default router
```

4. Работа с v-model, props, v-if/v-for, methods, mounted.

В форме NotesPage.vue поля input и textarea связаны двунаправленным связыванием v-model с объектом form (name, text, userId), а отправка формы обрабатывается через @submit.prevent="createCard". Список заметок notes получен из хранилища через computed-свойство (mapState), а методы loadNotes и createNote подключены через mapActions. Вывод карточек реализован директивой v-for по массиву заметок с передачей :name и :text в компонент NoteCard. В хуке жизненного цикла mounted() вызывается loadNotes(), подгружающий заметки при открытии страницы; при создании новой заметки (createCard) сначала вызывается createNote, затем повторно loadNotes для синхронизации с сервером, после чего форма сбрасывается через this.$refs.noteForm.reset(). В итоге получили такой проект (рис. 4 и 5)

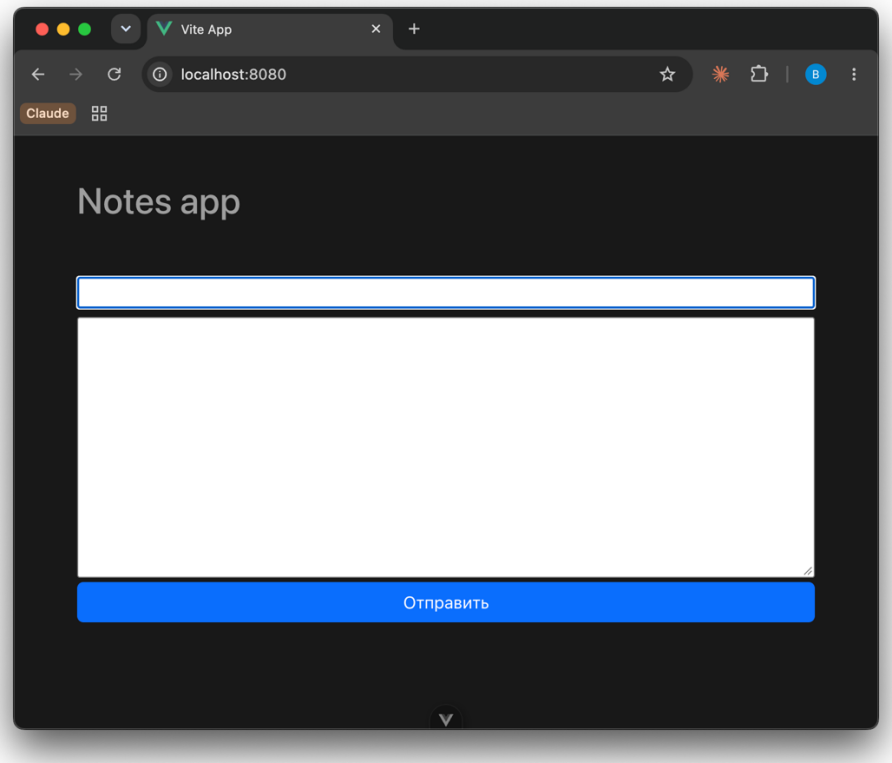


Рисунок 4 – Запущенный локально Notes app

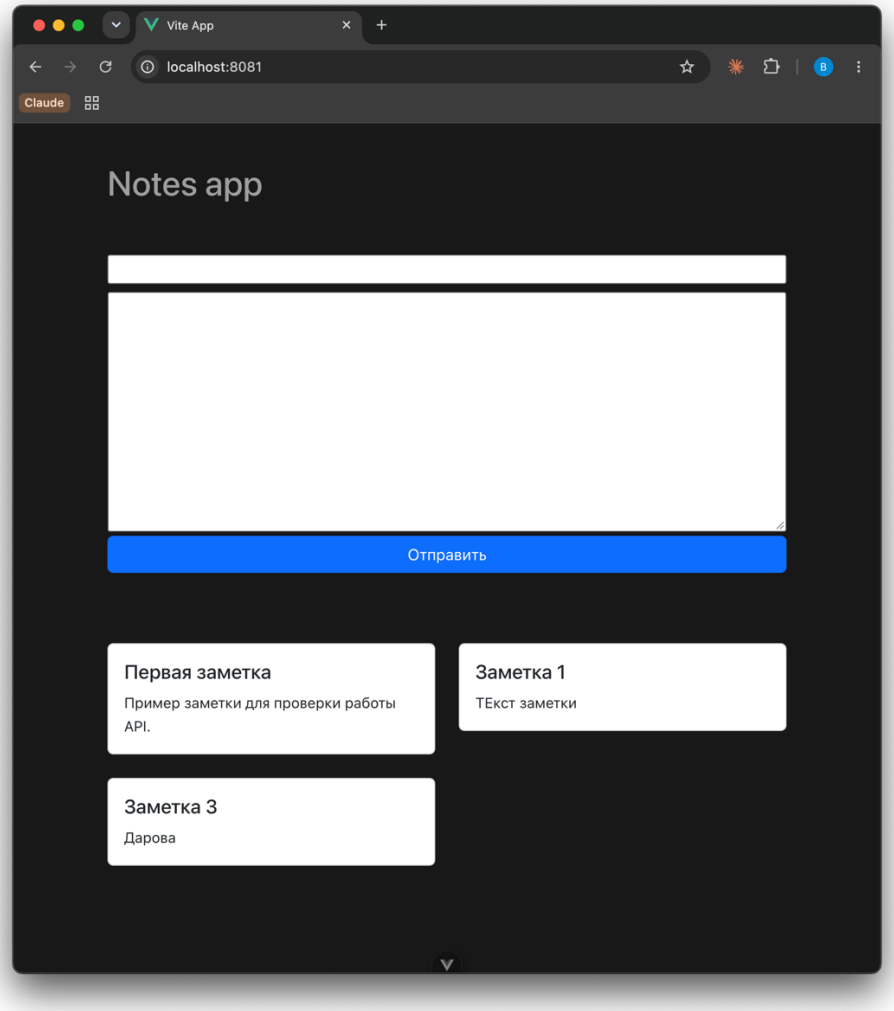


Рисунок 5 – Запущенный локально Notes app с добавленными заметками

5. Диагностика и устранение ошибок запуска.

При повторном запуске проекта командой "npm run dev" была обнаружена ошибка: "Cannot find native binding" для модуля @rolldown/binding-darwin-x64 – известная проблема npm, связанная с опциональными зависимостями (npm/cli#4828), из-за которой установленный Vite (на базе движка rolldown) не мог запуститься ни при каких обращениях к приложению. Проблема была устранена полным удалением node_modules и package-lock.json с последующей чистой установкой зависимостей ("npm install"), после чего dev-сервер Vite запустился корректно. Также было установлено, что в проекте пока отсутствует мок-сервер (json-server с db.json) на http://localhost:3000, поэтому запрос GET /notes из loadNotes() завершается сетевой ошибкой, а список заметок остаётся пустым до подключения backend-части API.

Вывод

В ходе выполнения домашней работы было создано Vue 3-приложение для ведения заметок с использованием Vite, подключён и настроен vue-router для организации навигации, реализована работа с внешним API через единый axios-инстанс по принципу внедрения зависимостей (IoC-контейнер в src/api/index.js), выполнено разумное разделение интерфейса на лейаут, представления и переиспользуемые компоненты, а также настроено управление состоянием через Pinia с сохранением данных в localStorage. В процессе работы были закреплены директивы v-model, v-for, обработчики событий и хуки жизненного цикла компонентов, а также получен опыт диагностики реальных ошибок окружения (конфликт нативных бинарных зависимостей Vite/rolldown) и понимание необходимости backend-части (json-server) для полноценной работы с API заметок.